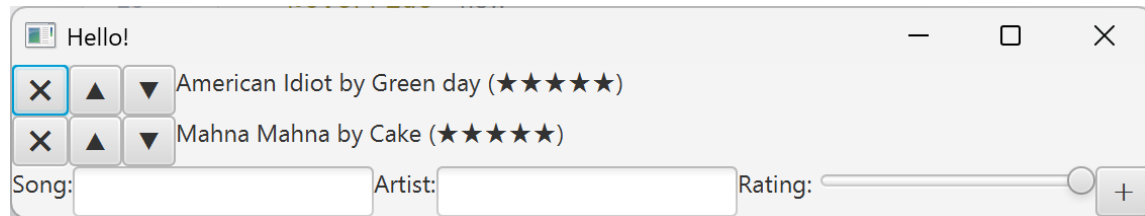


IC11: MVC - Front End



Today we will be collaborating with a partner (pair programming) to make a small fullstack application. One person is the front-end dev, the other the back-end dev. Pick a partner and pick your roles. You will each be contributing to the same repository, and pulling each other's work to get all of the code.

View

In this part of the in-class exercise, you will build the **View** for the playlist application. The View is responsible for rendering the user interface and handling user interactions by delegating actions to the Controller.

1 The FXComponent Interface

We use a common interface to ensure every visual component in our application can be rendered as a JavaFX Node.

We provide you the `FXComponent` interface, which contains the following method:

- `Parent render();`

You want to ensure that all classes in the View package (except the main application class) implement this interface.

2 View Class

] Like the 2048 example, it is recommended that you have a view class that will be a `ModelObserver` as well as an `FXComponent`. Its purpose will be to render all the views, and rerender the scene when the model updates.

You will also need to add this class as an observer to the Model.

3 The SongView Class

This class represents the visual display of a **single song** row within the playlist. Its purpose is to show the song's details and provide controls that allow the user to interact with that song. Your task is to create a concrete JavaFX component that keeps track of the necessary state and integrates properly with the controller.

- Create a class named `SongView` that implements `FXComponent`.
- Store the index of that song in the playlist, the model for reading the state, and the controller that handles user actions.
- Provide a constructor that accepts `Model model`, `int song_index`, and `Controller controller`, and use these values to initialize the instance variables.

Render Method

Implement the `render()` method to return a `Parent` (specifically a `Pane`). You should use an `HBox` layout to arrange elements horizontally.

Inside the `HBox`, add the following elements in order:

1. **Delete Button:** Label it with Unicode `"\u274C"` (x).
2. **Up Button:** Label it with Unicode `"\u25B2"` (up arrow). On action, call `controller.moveSongUp(song_index)`.
3. **Down Button:** Label it with Unicode `"\u25BC"` (down arrow). On action, call `controller.moveSongDown(song_index)`.
4. **Song Label:** A `Label` displaying the song info.

Each button should have a handler that correctly calls the corresponding method in the controller to perform the action.

Song Label requirements:

- The displayed text must follow the format `"Title by Artist (Rating)"`.
- The rating portion must display five symbols, using filled stars to represent the song's rating value and empty stars for the remaining positions.
 - For the rating, use a loop to append Unicode `"\u2605"` (filled star) for the rating value, and Unicode `"\u2606"` (unfilled star) for the remaining up to 5.

Hint. Feel free to make a helper method to modularize the code

It is recommended that you stop here, commit your work, and pull the backend progress for `SongImpl`. Wire up some dummy code either with hardcoded values in `SongView`, or an anonymous class `Controller` to try out your work.

4 The PlaylistView Class

The `PlaylistView` class is responsible for presenting the playlist to the user as part of the graphical interface. It constructs a visual representation of every song in the current model state and returns it as a JavaFX component.

Make a `View` that implements `FXComponent`. It's render method should:

- Iterate through every song in the model and
- Create and render a new `SongView`
- Return a vertical list of all songs.

It is suggested to stop here and merge your work. Add some songs to your model, and see if the `PlaylistView` can read and display them.

5 The AddSongView Class

The `AddSongView` class provides a form that allows users to enter information for a new song and submit it to the playlist. It serves as a view component that collects user input and forwards it to the controller.

- The class must function as an `FXComponent` that produces a JavaFX view.
- The view must include controls that allow users to input a song title, artist, and rating as follows:
 - A text input control for entering the song title.
 - A text input control for entering the artist name.
 - A slider that allows selecting a value from 0 through 5.
 - A button that submits the new song information.
- When the submission control is activated, the view must gather the input values and pass them to the controller so a new song can be added.

6 The Main View Class

You now have all the pieces to complete the `View` class.

Render Method

In `render()`, create a `VBox`. Add the rendered sub-panels to the `VBox` in this specific order:

1. `PlaylistView`
2. `AddSongView`

Update Method

Implement `void update(Mode playlist)`. Since the structure of the view might change (e.g., songs added/removed), you need to refresh the scene.

1. Create a new `Scene` using the result of `render()`.
2. Set this new scene on the `stage` (passed through the constructor).

7 Application

Set up your Model-View-Controller architecture in the start method. Be sure to add the view as an observer to the model, set your scene, and start the show!

8 Submission

When you are finished, or when class ends (which ever comes first), commit your current changes to github, and submit what you have to gradescope!